
Pie-Lab 2025年暑期培训

李秋予

Pie-Lab

北京理工大学计算机学院
北京市海淀区中关村南大街
786362411@qq.com

Abstract

以下是使用自己录制的数据,配置Gaussian Splatting环境,3. 使用Colmap生成初始点云,训练场景的高斯模型,并实现新视角图像生成的实验报告,包括Gaussian Splatting 与 NeRF 的区别分析、球谐函数数学背景与实现细节、Adaptive Density Control控制方法以及Fast Differentiable Rasterizer的加速思路。

1 Gaussian Splatting 与 NeRF 的区别分析

Neural Radiance Fields (NeRF) 和 3D Gaussian Splatting 是两种用于3D场景重建和渲染的技术。它们都旨在创建高质量的3D图像,但它们的技术原理和应用场景有所不同。

1.1 数据输入 (input)

NeRF: Nerf的输入是一张图像+该图像对应的相机位姿5D输入 (xyz和与)

3DGS: 3DGS的输入是由一张图像+经过SFM方法后生成的稀疏点云

比较: 两者的差距在于一个是注重于相机的位姿(观测者的姿态)一个是注重于图像中各点的位置(2D图像转换为3D点云观测内容的姿态)

Nerf的5D位姿会进行一个正弦编码才会输入到MLP网络中进行运算,这个编码过程让网络能够学习到更多的高频数据,从而提升了网络对于图像细节的重建能力,如果把这个过程看作input数据的初始化,那么与之对比,3DGS的输入在得到稀疏点云之后同样进行了初始化,将稀疏点云建模为了3D高斯,才进行后续的处理,因此两者在初始化阶段也有一些不同。

1.2 数据输出 (output)

NeRF: NeRF的输出是经由神经网络之后直接输出对应camera ray上各个采样点的RGB值和体密度(volume density),随后经过体渲染(volume rendering)得到最终的重建图像。

3DGS: 3DGS的输出,是直接将最终重建得到的图像作为了整个方法的最终输出。

比较: 两者的最终输出都是重建图像,但是获得重建图像的过程中存在差异,即两者的渲染方式不同。NeRF的渲染方式是体渲染,神经网络输出camera ray上不同采样点的RGB和体密度后,对该camera ray进行一个accumulate,最终得到2D图像。3DGS在会在过程中将建模好的3D高斯进行光栅化处理,将其投影到2D图像中,这个过程可以理解为向一个平地抛雪球,这个雪球被染上了不同的颜色,雪球落地之后会溅开,产生不同颜色的痕迹,多个雪球(3D高斯)丢到平地上之后,把不同的颜色痕迹按照深度等进行混合,就得到了最终的图像,这个渲染过程就叫做splatting,这里的渲染技术是a-blending。

a-blending这种方法可以将一个对象的颜色与其背景颜色按照特定的比例混合,从而实现透明效果。这是通过使用一个额外的a通道(透明度通道)来实现的,它决定了图像中每个像素

的不透明度。a-blending广泛应用于二维和三维图形渲染中，用于创建更加逼真的视觉效果，如玻璃、烟雾、水面等透明或半透明的物体。

1.3 三维信息表达(3D Information Expression)

NeRF：作为神经辐射场，其三维信息藏在MLP网络中，而神经网络具有不可解释性，类似于一个黑匣子，因此神经辐射场常常和隐式等词一起出现，就是因为其重建三维世界的信息是隐式表达。

3DGS：由SFM建立得到的稀疏三维点云，在建模成3D高斯后，3D高斯中明确的包含了三维世界的信息（位置，颜色，不透明）因为3D高斯的前身是三维点云，而三维点云是典型的显示表达数据，因此3DGS是一种显示表达。

比较：NeRF是隐式表达，三维信息藏在神经网络中，3DGS是显示表达，由3D高斯直接表达三维信息。

1.4 优化方式

两者的优化方案在思路都是相同的，都是将渲染出来的图像和真实图像进行对比，求loss，然后不断最小化这个loss来优化过程中的各个参数变量，都采用了经典的梯度下降优化策略。

2 球谐函数数学背景与实现细节

2.1 定义与性质

球谐函数（Spherical Harmonics, SH）是数学中的一类特殊函数，它们在三维空间中用于描述依赖于方向的物理现象。这些函数在物理学、地球物理学、天文学、计算机图形学、量子力学等领域有着广泛的应用。

球谐函数是一组正交函数，定义在单位球面上。它们可以被看作是二维傅里叶级数在球面坐标系中的等价物，用来表示球面上的任何平方可积函数。球谐函数通常用 $Y_l^m(\theta, \phi)$ 表示，其中 l 是度（degree）， m 是阶（order），而 θ 和 ϕ 分别是极角（从 z 轴测量的角度）和方位角（从 x 轴测量的角度）。

球谐函数的形式如下：

$$Y_l^m(\theta, \phi) = K_l^m P_l^m(\cos \theta) e^{im\phi} \quad (1)$$

这里：

- K_l^m 是一个归一化因子。
- P_l^m 是关联勒让德多项式（Associated Legendre Polynomials）。
- $e^{im\phi}$ 表示复指数函数，它提供了方位角 ϕ 的周期性。

球谐函数有以下特性：

1. **正交性**：不同 l 或 m 值的球谐函数之间是正交的。
2. **完备性**：所有可能的球谐函数构成一个完备集，意味着任何一个定义在球面上的平方可积函数都可以通过球谐函数的线性组合来表示。
3. **对称性**：球谐函数具有不同的对称性质，这取决于它们的 m 和值。

2.2 球谐函数

球谐（Spherical Harmonics）函数是一组基函数，通常用于表示球面上的函数。在计算机图形学中，球谐函数被广泛应用于光照和反射模型中，用来表示光照或反射的强度分布。

球谐函数，归根到底是一组基函数，有了基函数，就可以把任意一个函数，描述成几个基函数的加权和了，一般的，能用的基函数个数越多，表达能力就越强。

2.3 基函数

基函数其实是一个相对的概念，类似于空间中的基向量，基向量可以通过组合表示张成的空间中的所有向量。同理，基函数也可以通过线性or非线性组合来表示其张成的函数空间的函数，傅里叶变换、球面谐波都是常见的基函数线性组合为原函数。

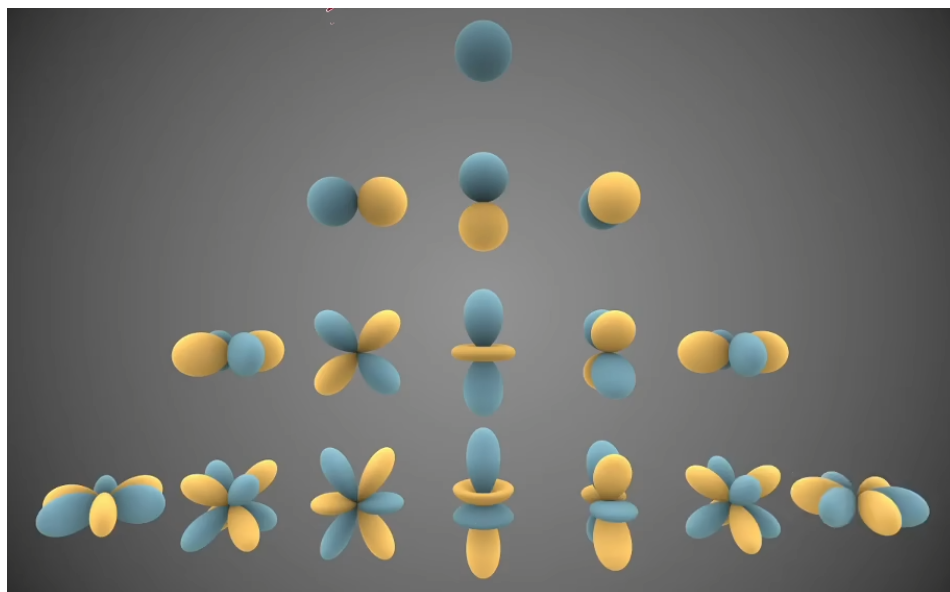


Figure 1: SH的基函数

当SH的系数用的越多，那么表达能力就越强，跟原始的函数就越接近。

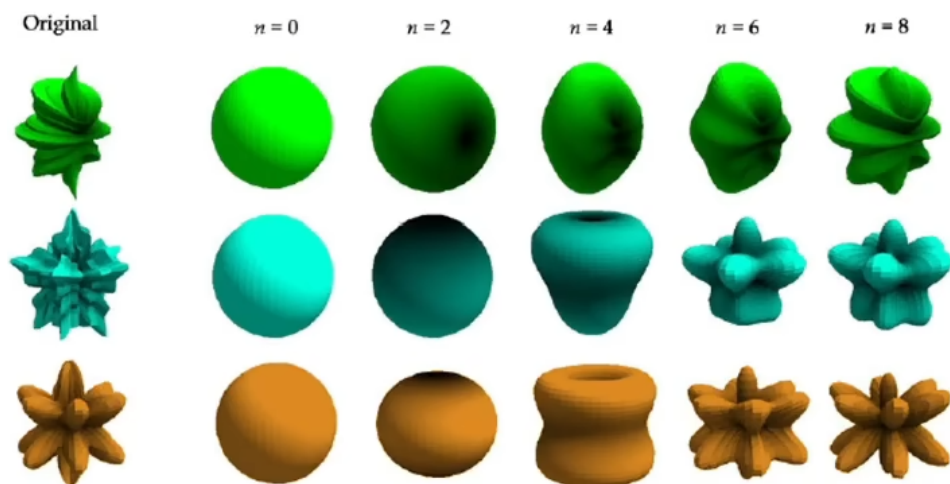


Figure 2: 球谐函数不同系数示例

3 Adaptive Density Control控制方法

使用SfM算法初始化了一系列稀疏点之后，adaptive densification方法会动态调整3D Gaussians的数量和密度。因为初始化点云质量不高，不适合完全依赖其初始化点云，所以要优化点和点数，它包含两种方法：

1. Pruning减少伪影的出现：

其中不透明度低于设置的阈值或者离相机近的一些点会进行删除

2. Densification 过度重构或者欠采样(基于梯度变化来判断):

方差很小 $\rightarrow$ 克隆高斯来适应 Under-Reconstruction

方差很大 $\rightarrow$ 分割成两个高斯 Over-Reconstruction

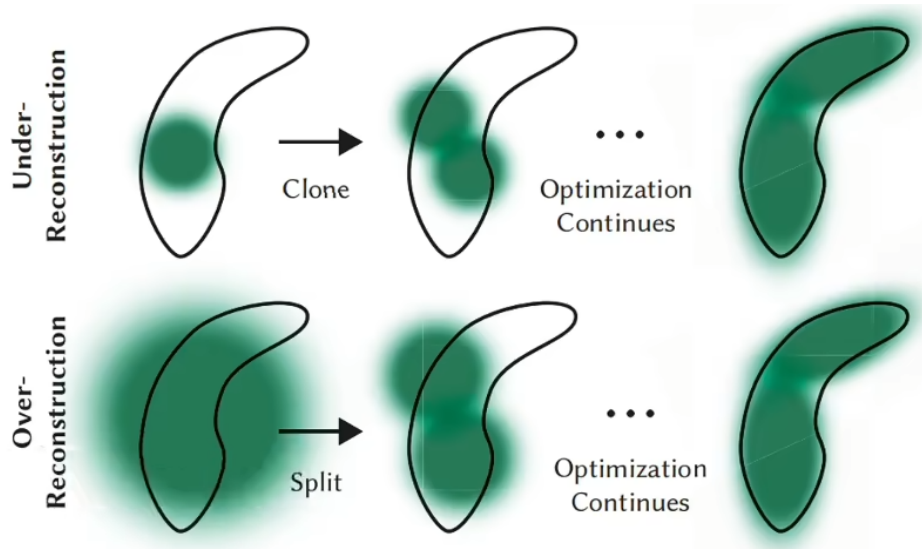


Figure 3: Adaptive Density Control

4 Fast Differentiable Rasterizer的加速思路

假设现在已经得到了用于表示整个场景的大量3D Gaussian，现在我们要将其渲染到image上。

为了加速渲染，3DGS选择使用Tile-based rasterization，将Image切为一个个 $16 * 16$ 的tile，每个tile像一个小image一样独立计算颜色值，最后多个Tile拼成image的颜色。

考虑到每个3D Gaussian投影到2D image时可能会投影到多个tile，处理方法是如果一个Gaussian投影与多个tile相交，就实例化多次Gaussian，每个Gaussian instance都会得到一个结合view space depth和tile ID得到的key。然后基于这些key，使用single fast GPU Radix sort进行排序。

如下图中，黄色Gaussian投影影响了Tile1和2，其他Gaussian投影同理；在另一张图中，我们给出了一个Gaussian投影到多个Tile后，多次实例化以及排序的操作。

进行排序的目的以及意义：

1. 确保正确的深度排序

在光栅化渲染中，像素的最终颜色由多个重叠的高斯混合决定，而混合顺序必须遵循从远到近（Back-to-Front）的深度顺序（即Alpha混合的over操作）。如果高斯的渲染顺序错误，会导致透明效果异常（如本应被遮挡的高斯错误覆盖前景）。

每个高斯实例的key由两部分组成：

(1) Tile ID：标识该高斯属于哪个Tile。

(2) View Space Depth：高斯中心在相机空间的深度值（通常取投影后的 z 值）。

基数排序会按Tile ID + Depth对高斯实例进行排序，结果保证同一Tile内的高斯按深度从远到近排列，并且不同Tile的高斯彼此隔离（Tile间无需排序）。

2. 优化GPU内存访问

GPU的并行计算效率高度依赖内存访问的连续性，如果高斯在内存中无序存储，线程访问会是随机的，导致显存带宽利用率低下。

基数排序后，同一Tile的高斯在内存中连续排列，线程块（CUDA Block）处理一个Tile时，所有需要的高斯数据集中存储，并减少缓存失效（Cache Miss），提升数据加载速度。



Figure 4: 16 * 16的tile

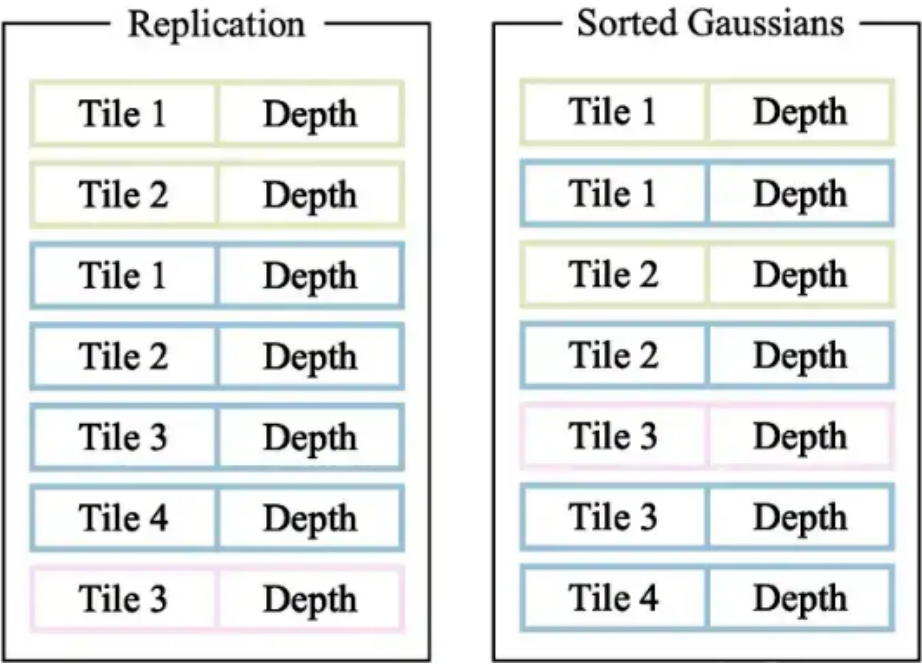


Figure 5: 排序结果

之后，每个Tile分别进行Alpha Blending,分而治之，计算像素颜色值得到图像。

5 效果可视化展示

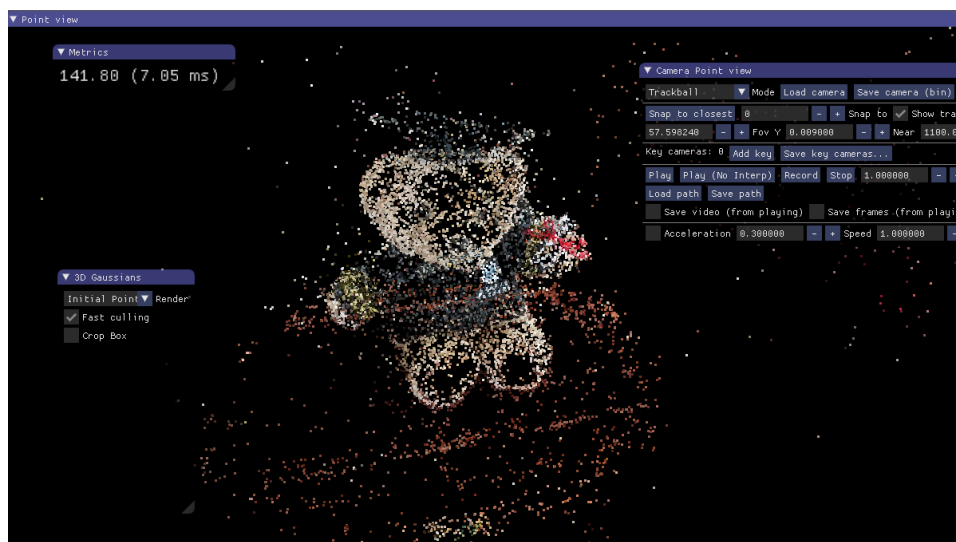


Figure 6: 初始化点云分布情况

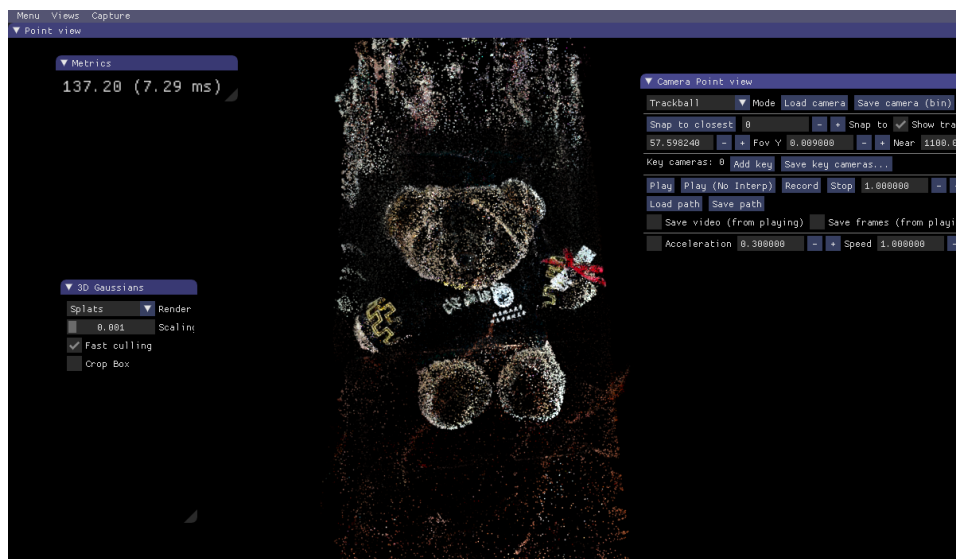


Figure 7: 执行30000步时点云分布情况

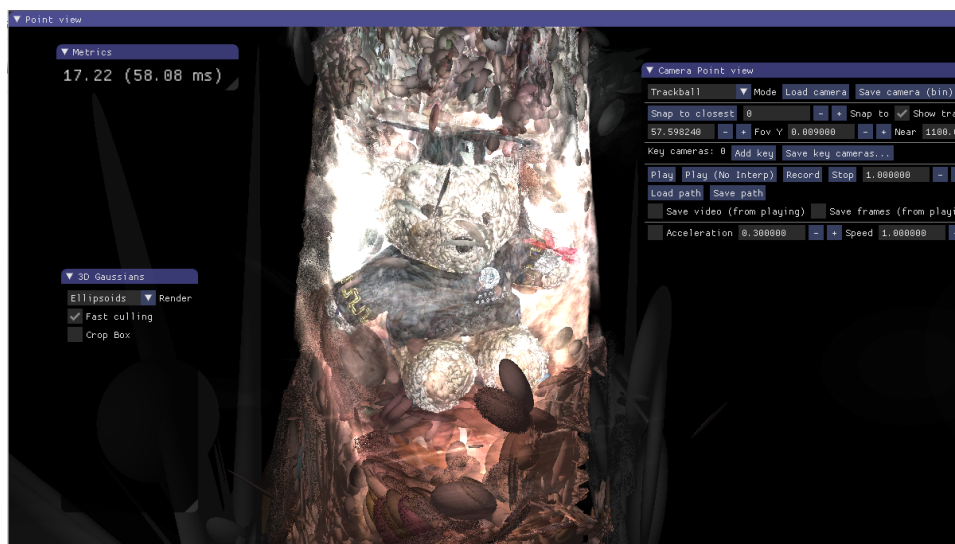


Figure 8: 椭圆高斯分布情况

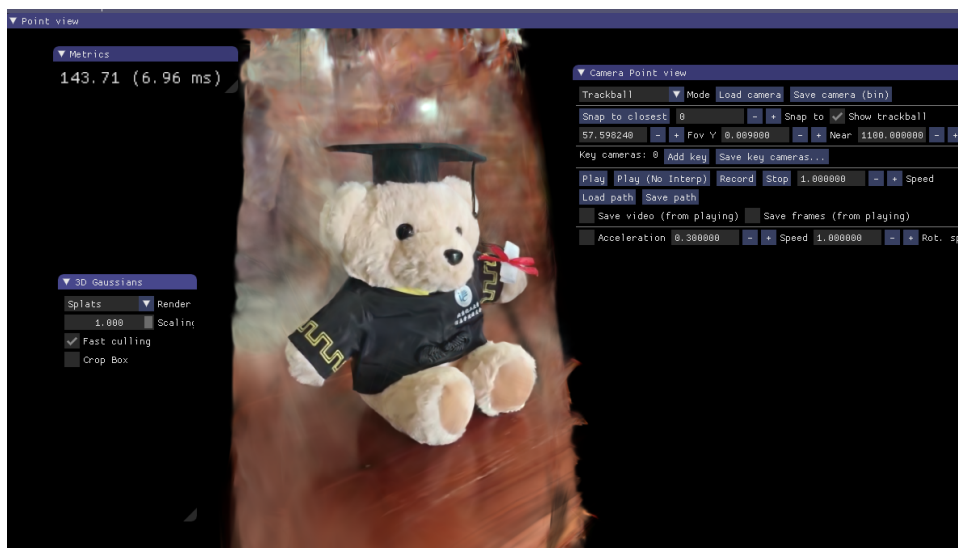


Figure 9: 最终结果展示